Design of Software Systems -CS612

GROUP PROJECT

Canteen Pre-Order System

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Architecture Style: Service-Oriented Architecture (SOA)

Group - 4 Team Members

Name	Role
Syed Mohammed Umar (2621532)	Project Lead & Backend Developer
Kaustav Raj Chakravorty (2621516)	Design & Development Team Lead, Frontend, Backend Developer
Hrishiraj Gogoi (2621513)	Requirement Analyst
Akash Kumar Yadav (2621505)	Designer

Table of Contents

1. Project Proposal **4**
1.1 Project Title 4
1.2 Proposal 4
2. Introduction **4**
3. Problem Statement **5**
4. Objectives **6**
5. Scope of the System **6**
6. Target Users **7**
6.1 Student 7
6.2 Faculty 7
6.3 Canteen Staff 8
6.5 System Administrator 8
7. Major System Modules **9**
8. Overall Use Case **9**
9. Major Entities **11**
10. Functional Requirements **13**
10.1 Authentication 13
10.2 User Management 13
10.3 Menu Management 13
10.4 Inventory 14
10.5 Cart 14
10.6 Pre-Order 14
10.7 Pickup Slots 14
10.8 Payment 15
10.9 Order Processing 15
10.10 Pickup Verification 15
10.11 Cancellation 15
10.12 Notifications 16
11. Non-Functional Requirements **16**
11.1 Performance 16
11.2 Reliability 16
11.3 Security 16
11.4 Data Integrity 17
11.5 Usability 17
11.6 Scalability 17
11.7 Maintainability 17

11.8 Interoperability..... 18
11.9 Privacy..... 18
11.10 Backup and Recovery..... 18
12. Technology Stack..... 18
13. Database Entities..... **22**
14. External Interfaces..... **22**
14.1 User Interface..... 22
14.2 Payment Interface..... 22
14.3 Notification Interface..... 22
15. Constraints..... **22**
16. Assumptions..... **23**
17. Project Team Roles and Responsibilities..... **23**
17.1 Syed Mohammed Umar (2621532)..... 23
Project Lead & Backend Developer..... 23
17.2 Kaustav Raj Chakravorty (2621516)..... **24**
Design & Development Team Lead, Frontend & Backend Developer..... 24
17.3 Hrishiraj Gogoi (2621513)..... **24**
Tester..... 24
17.4 Akash Kumar Yadav(2621505)..... 25
Designer-System Architecture & Workflow..... 25
18. Development and Testing Strategy..... **25**
Phase 1: Requirement Analysis & Design..... 25
Phase 2: Core Development..... 26
Phase 3: Integration..... 26
Phase 4: Testing..... 26
Phase 5: Deployment & Demonstration..... 26
19. Future Enhancements..... **27**
20. Expected Outcome..... **27**
21. Conclusion..... **28**

1. Project Proposal

1.1 Project Title

Canteen Pre-Order System

1.2 Proposal

The proposed **Canteen Pre-Order System** is a software application designed to allow students, faculty, and staff to browse the canteen menu, check real-time food availability, place food orders in advance, select convenient pickup time slots, make payments, and collect their orders using a unique order identifier.

The system aims to reduce long queues, improve order management, minimize waiting time, and provide canteen staff with a structured mechanism for handling incoming orders.

Unlike a conventional food-ordering application, the proposed system is specifically designed around a pre-order model suitable for an academic campus environment. The system will manage food availability, inventory quantities, order processing, payment status, notifications, and order collection.

The application will follow a **Service-Oriented Architecture (SOA)** in which major business capabilities are separated into logically independent services that communicate through well-defined REST APIs.

---

2. Introduction

2.1 Background

College and university canteens frequently experience high demand during short periods such as breakfast, lunch, and evening breaks. Students and faculty may spend considerable time waiting in queues to place orders and collect food.

At the same time, canteen staff have to manually manage incoming orders, food availability, preparation, and customer collection. This can lead to congestion, inefficient order handling, incorrect orders, and difficulty in estimating demand.

The proposed Canteen Pre-Order System provides a centralized digital platform through which customers can place orders before reaching the canteen.

The system will allow customers to:

- Browse the available menu.
- Check food availability.
- Add items to a cart.
- Select quantities.
- Place an order.
- Make or simulate payment.
- Track order status.
- Receive notifications.

Canteen personnel will receive and process orders through dedicated interfaces.

---

3. Problem Statement

The existing manual or semi-digital canteen ordering process can result in:

- Long queues during peak hours.
- Increased waiting time.
- Difficulty in knowing food availability before reaching the counter.
- Manual order-processing errors.
- Difficulty managing large numbers of simultaneous orders.
- Poor visibility of order status.
- Inefficient management of pickup times.
- Difficulty maintaining accurate inventory.
- Limited operational reports for canteen management.

Therefore, a centralized pre-ordering system is proposed to digitally coordinate customers, food inventory, orders, payments, preparation, and pickup.

---

4. Objectives

The major objectives of the system are:

1. To provide a digital platform for pre-ordering food from the campus canteen.
2. To reduce customer waiting and queue time.
3. To provide real-time or near-real-time food availability.
4. To provide canteen staff with organized order-processing functionality.
5. To maintain consistent food inventory and order information.
6. To provide order tracking from placement to collection.
7. To provide basic payment functionality.
8. To provide reports and analytics for canteen management.
9. To demonstrate the application of **Service-Oriented Architecture** in a real-world software system.
10. To provide a modular system that can be extended with additional services in the future.

---

5. Scope of the System

5.1 In Scope

The system will include:

- User registration and authentication.
- Customer profile management.
- Menu browsing.
- Food search and filtering.
- Food availability management.
- Shopping cart.
- Pre-order placement.
- Order management.
- Order-status tracking.
- Payment/mock payment processing.
- Pickup verification.
- Order cancellation where applicable.
- Notifications.
- Feedback and ratings.
- Inventory management.

- Canteen management dashboard.
- Administrative functions.

---

6. Target Users

The system will have the following primary users.

6.1 Student

Students can:

- Register/login.
- Browse food.
- Check availability.
- Add food to cart.
- Place pre-orders.
- Make payment.
- Track orders.
- Receive notifications.
- Collect orders.
- View order history.
- Provide feedback.

6.2 Faculty

Faculty and staff can perform customer-related activities including:

- Register/login.
- Browse food.
- Check availability.
- Add food to cart.
- Place pre-orders.
- Make payment.
- Track orders.
- Receive notifications.
- Collect orders.
- View order history.

- Provide feedback.

6.3 Canteen Staff

Canteen staff can:

- View incoming orders.
- Accept orders.
- Prepare orders.
- Update order status.
- Verify customer pickup.
- Mark orders as collected.
- Update food availability.
- Review customer feedback.

6.5 System Administrator

The administrator can:

- Manage users.
- Manage roles.
- Manage permissions.
- Update prices.
- Manage inventory.
- Monitor sales.
- View reports.

---

7. Major System Modules

The system will consist of the following major modules/services:

1. Authentication & User Service
2. Menu Service
3. Inventory Service

4. Order Service
5. Payment Service
6. Notification Service
7. Feedback Service
8. Reporting Service
9. Administration Service

---

8. Overall Use Case

Use Case: Student Places a Pre-Order

Actor

Student / Customer

Preconditions

- The customer has an active account.
- The customer is authenticated.
- The selected food items are available.

Main Flow

1. The student logs into the system.
2. The system authenticates the student.
3. The student opens the menu.
4. The system retrieves available food items from the Menu Service.
5. The student selects a food item.
6. The system retrieves current availability from the Inventory Service.
7. The student adds the item to the cart.
8. The student selects the required quantity.
9. The system displays the order summary.
10. The student confirms the order.
11. The Order Service creates the order.
12. The Inventory Service reserves the required quantity.
13. The Payment Service processes or simulates payment.
14. The system confirms the order.
15. A unique order ID is generated.

16. The Notification Service sends an order confirmation.
17. Canteen staff view the new order.
18. Staff accept and prepare the order.
19. Staff mark the order as **READY**.
20. The customer receives a notification.
21. The customer reaches the canteen and verifies the order.
22. The system marks the order as **COLLECTED**.

Exceptional Case:

If payment fails:

Payment Failed
↓
Order remains unconfirmed
↓
Reserved inventory is released
↓
Customer is informed

If the selected food becomes unavailable:

Availability Check
↓
Food unavailable
↓
Order cannot be confirmed
↓
Customer selects another item

---

9. Major Entities

User

Attributes:

- User ID
- Name
- Email

- Password hash
- User type
- Account status

Food Item

Attributes:

- Food ID
- Name
- Description
- Category
- Price
- Availability
- Quantity

Order

Attributes:

- Order ID
- User ID
- Order date
- Total amount
- Status
- Pickup slot
- Payment status

Order Item

Attributes:

- Order Item ID
- Order ID
- Food ID
- Quantity
- Price

Pickup Slot

Attributes:

- Slot ID
- Date
- Start time
- End time
- Maximum capacity
- Current booking count

Payment

Attributes:

- Payment ID
- Order ID
- Amount
- Payment method
- Payment status
- Transaction reference

Notification

Attributes:

- Notification ID
- User ID
- Message
- Type
- Timestamp
- Status

Feedback

Attributes:

- Feedback ID
- User ID
- Order ID
- Rating
- Comment
- Timestamp

10. Functional Requirements

10.1 Authentication

1. The system shall allow users to register.
2. The system shall allow registered users to log in.
3. The system shall authenticate users before accessing protected functions.
4. The system shall allow users to log out.
5. The system shall provide password recovery functionality.

10.2 User Management

1. The system shall allow users to view their profiles.
2. Users shall be able to update permitted profile information.
3. The system shall maintain user order history.
4. Administrators shall be able to manage user accounts in a protected capacity.

10.3 Menu Management

1. The system shall allow customers to view the menu.
2. The system shall organize food items into categories.
3. The system shall display food name, description, price, and availability.
4. Customers shall be able to search for food items.
5. Customers shall be able to filter food items.
6. Admin shall be able to add food items.
7. Admin shall be able to modify food items.
8. Admin shall be able to remove food items.

10.4 Inventory

1. The system shall maintain food quantities.

2. Staff shall be able to update food availability.
3. The system shall automatically mark items as unavailable when quantity reaches zero.
4. The system shall prevent customers from ordering unavailable quantities.
5. The system shall update inventory following successful orders.

10.5 Cart

1. Customers shall be able to add food items to a cart.
2. Customers shall be able to change quantities.
3. Customers shall be able to remove items.
4. The system shall calculate the cart subtotal and total.

10.6 Pre-Order

1. Customers shall be able to place food pre-orders.
2. The system shall verify food availability before confirmation.
3. The system shall generate a unique order ID.
4. The system shall provide order confirmation.

10.7 Payment

1. The system shall display the final payable amount.
2. The system shall process or simulate payment.
3. The system shall record payment status.
4. The system shall associate payment information with the corresponding order.

10.8 Order Processing

1. Canteen staff shall be able to view incoming orders.
2. Staff shall be able to accept orders.
3. Staff shall be able to update preparation status.
4. The system shall maintain order states such as:
5. PLACED → ACCEPTED → PREPARING → READY → COLLECTED

6. Customers shall be able to view the current status of their order.

10.9 Pickup Verification.

1. Staff shall be able to verify pickup via order ID.
2. The system shall mark an order as collected after successful verification.
3. The system shall prevent an already collected order from being collected again.

10.10 Cancellation

1. Customers shall be able to cancel eligible orders.
2. The system shall apply the configured cancellation policy.
3. The system shall update inventory following successful cancellation.
4. The system shall record cancellation information.

10.11 Notifications

1. The system shall notify customers after order placement.
2. The system shall notify customers after successful payment.
3. The system shall notify customers when orders are ready.
4. The system shall notify customers of cancellation or rejection.

---

11. Non-Functional Requirements

11.1 Performance

1. The system shall provide responsive performance during normal usage.
2. The system shall support multiple users accessing the system concurrently without significant degradation of core functionality.

3. The system shall handle concurrent order requests while maintaining order and inventory consistency.

11.2 Reliability

1. The system shall prevent duplicate order creation caused by repeated requests.
2. Confirmed orders shall not be lost during normal system operation.
3. The system shall maintain consistency between orders and inventory.

11.3 Security

1. The system shall implement role-based authorization.
2. Passwords shall not be stored in plain text.
3. Sensitive communication shall use HTTPS in production.
4. The system shall validate user input.

11.4 Data Integrity

1. The system shall prevent multiple users from successfully consuming the same inventory quantity.
2. Payment and order records shall remain consistent.
3. Historical order records shall be preserved.

11.5 Usability

1. The customer interface shall be simple and intuitive.
2. Food availability, price, quantity and pickup time shall be clearly displayed.
3. Error messages shall provide understandable information.
4. The application shall support desktop and mobile screen sizes.

11.6 Scalability

1. The architecture shall permit additional services to be added without major modification to existing services.
2. The system shall support expansion to additional canteens in future versions.

11.7 Maintainability

1. Services shall have clearly defined interfaces.
2. The system shall use modular software components.
3. APIs shall be documented.
4. Source code shall follow consistent coding standards.

11.8 Interoperability

1. Services shall communicate using REST APIs.
2. JSON shall be used as the primary data exchange format.
3. External payment and notification services shall be accessed through defined interfaces.

11.9 Privacy

1. The system shall collect only required user information.
2. Users shall only be able to access their own personal and order information.
3. Payment credentials shall not be stored directly by the application where an external payment provider is used.

11.10 Backup and Recovery

1. Critical system data shall be backed up periodically.
2. The system shall provide a mechanism for restoring data from a valid backup.

---

12. Technology Stack

Frontend

React Native

Used for:

- Customer interface
- Canteen staff interface
- Administrator interface.

Backend

Python

Used to develop the individual SOA services and APIs.

API Communication

RESTful APIs

Used for communication between the frontend, API gateway and backend services.

Database

PostgreSQL

Used for:

- User data
- Menu
- Inventory
- Orders
- Payments
- Pickup slots
- Feedback
- Notifications
- Audit records

ORM

SQLAlchemy

Used for database interaction.

Authentication

JWT Tokens

Used for authentication and role-based authorization.

Payment

Mock Payment Service / Payment Sandbox

A simulated payment service can be used for the academic prototype.

Notifications

Email/SMTP and in-application notifications

Used for order and status notifications.

Testing

- Postman

Version Control

Git + GitHub

Used for collaborative development and version management.

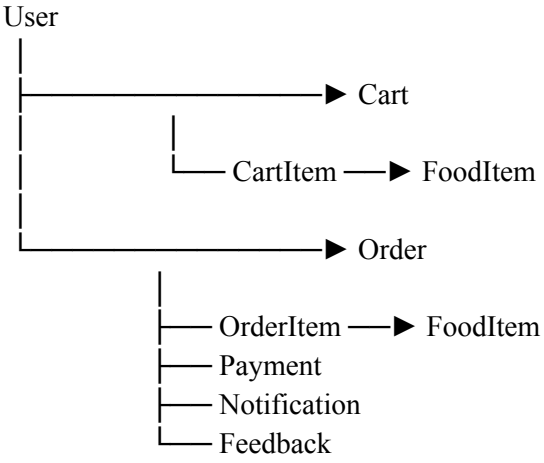
---

13. Database Entities

The primary database entities will include:

/cart_items
/feedback
/food_images
/food_items
/Inventory
/notification
/order_events
/Orders
/password_resets
/Payments
/roles
/users

Basic relationship flow



---

14. External Interfaces

14.1 User Interface

The system shall provide web-based interfaces for:

- Customers
- Canteen staff
- Administrators

14.2 Payment Interface

The payment service will communicate with a payment gateway or mock payment service through APIs.

14.3 Notification Interface

The system may communicate with an email or notification provider.

---

15. Constraints

The project is an academic prototype and therefore has the following constraints:

1. Development time is limited.
2. The system will initially support a limited number of users.
3. Payment may use a mock/sandbox implementation.
4. Real-world payment settlement is outside the initial scope.
5. Hardware integration is outside the initial scope.
6. External services may be replaced by simulated services during demonstrations.
7. Deployment resources may be limited.

---

16. Assumptions

The system assumes that:

1. Users have access to a valid account.

2. The canteen maintains accurate food availability information.
3. Canteen staff regularly update order status.
4. Users have access to an internet connection.
5. Payment services, where used, are available.
6. The system is deployed in an environment capable of running the required backend services and database.

---

17. Project Team Roles and Responsibilities

17.1 Syed Mohammed Umar (2621532)

Project Lead & Backend Developer

Primary Responsibilities:

- Coordinate overall project development, milestones, and deliverables.
- Analyze requirements and contribute to the system architecture and SOA design.
- Design backend service boundaries, APIs, and database relationships.
- Develop backend services using Python and FastAPI.
- Implement authentication, authorization, and User/Authentication Service.
- Contribute to Order Service and backend integration.
- Coordinate final system integration, debugging, and deployment.
- Maintain SRS, architecture, and backend documentation.

17.2 Kaustav Raj Chakravorty (2621516)

Design & Development Team Lead, Frontend & Backend Developer

Primary Responsibilities:

- Coordinate day-to-day development activities and maintain coding standards.
- Design and develop the customer-facing web application and staff/manager interfaces.
- Implement menu browsing, search, cart, checkout and order tracking

- Develop backend services including Menu, Inventory.
- Contribute to Order Service and Payment/Notification integration.
- Integrate frontend components with backend REST APIs.
- Assist with Docker-based integration, debugging, and deployment.

17.3 Hrishiraj Gogoi (2621513)

Analyst

Primary Responsibilities:

- Requirement Gathering: Identified and gathered the functional and non-functional requirements of the Canteen Pre-order System.
- Stakeholder Identification: Identified the major actors - Students/Customers, Canteen Staff and Admin-and understood their respective needs.
- SRS Documentation: Prepared the Software Requirements Specification(SRS), documenting the system's scope, objectives, requirements and constraints.
- Use case and Requirement Analysis: Analysed the system workflows, use cases and functional requirements, including user authentication, menu management, cart, pre-orders, payments and order tracking
- Coordination with developers: Communicated requirements to the development team, clarified expected system developer and helped ensure that the implementation aligned with the documented requirements.

17.4 Akash Kumar Yadav (2621505)

Designer – System Architecture & Workflow

Primary Responsibility: Design and documentation of the system architecture, workflows, and UML/use-case representations.

System Architecture Responsibilities

- Design the overall **Service-Oriented Architecture (SOA)** of the system in coordination with the team.
- Identify and represent major services and their interactions.
- Design the high-level system architecture diagram.

- Define the interaction flow between frontend, API Gateway, SOA services, database, and external services.
- Assist in defining service boundaries and responsibilities.
- Review architectural consistency throughout the development process.
- Contribute architecture and design sections of the SRS.
- Maintain system architecture documentation.
- Ensure diagrams are updated whenever major system requirements or architectural decisions change.
- Coordinate with the development team to ensure that diagrams accurately reflect the implemented system.

18. Development and Testing Strategy

The project will follow an iterative development approach.

Phase 1: Requirement Analysis & Design

- Requirement gathering
- SRS
- Use cases
- UML diagrams
- Architecture design
- Database design

Phase 2: Core Development

- Authentication
- Menu
- Inventory
- Order
- Cart
- Pickup slots

Phase 3: Integration

- Payment
- Notifications
- Feedback
- Reporting

Phase 4: Testing

- API testing
- Manual Testing

Phase 5: Deployment & Demonstration

- Final integration
- Demonstration
- Presentation

---

19. Future Enhancements

Future versions could include:

1. A future version may introduce a surplus-food tray for cancelled prepared orders, offering food free or at a discount to reduce wastage.
2. Real payment gateway integration.
3. Multiple campus canteens.
4. AI-powered food recommendations.
5. Natural-language food search.
6. Demand forecasting.
7. Personalized recommendations.
8. Automated inventory prediction.
9. Loyalty/reward system.
10. Promotional offers.
11. Advanced analytics.
12. Integration with institutional identity systems.

---

20. Expected Outcome

The final system is expected to provide a functional web-based platform through which campus users can conveniently pre-order food and collect it during a selected time slot.

The system will demonstrate:

1. Practical application of SOA.
2. Modular service design.
3. REST-based service communication.

4. Database-driven transaction management.
5. Concurrent order and inventory handling.
6. Role-based access control.
7. Order lifecycle management.
8. Payment processing.
9. Notification mechanisms.
10. Software testing and quality assurance.

---

21. Conclusion

Our **Canteen Pre-Order System** is proposed as a modular software solution for improving the efficiency of campus food ordering and pickup. The system addresses common problems such as queues, uncertain food availability, manual order management, and inefficient pickup processes.

By implementing the system using **Service-Oriented Architecture**, our project demonstrates how independent software services can collaborate to provide a complete application. The architecture also allows individual services such as ordering, inventory, payment, notifications, and pickup management to be developed, tested, and extended independently.

The proposed technology stack of React, JavaScript, PostgreSQL, REST APIs, Docker, GitHub, and automated testing tools provides a practical foundation for implementing the system as an academic M.Tech project.

The initial implementation will focus on the core pre-ordering workflow while maintaining an architecture that can support future enhancements such as AI recommendations, demand forecasting, mobile applications, and multi-canteen support.